In [7]:
```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
```

In [1]:
```
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt to /home/csl-4/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /home/csl-4/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /home/csl-4/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package wordnet to /home/csl-4/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Out[1]:
```
True
```

In [2]:
```
# Sample document
text = "Text analytics is the process of extracting meaningful information fr
```

In [3]:
```
print("\nOriginal Text:\n", text)
```

```
Original Text:
 Text analytics is the process of extracting meaningful information from unst
ructured text data using natural language processing techniques.
```

In [5]:
```
from nltk.tokenize import word_tokenize
```

In [6]:
```
# Tokenization
tokens = word_tokenize(text)
print("\n1. Tokens:\n", tokens)
```

```
1. Tokens:
 ['Text', 'analytics', 'is', 'the', 'process', 'of', 'extracting', 'meaningfu
l', 'information', 'from', 'unstructured', 'text', 'data', 'using', 'natura
l', 'language', 'processing', 'techniques', '.']
```

In [8]:
```
pos_tags = pos_tag(tokens)
print("POS Tags:", pos_tags)
```

```
POS Tags: [('Text', 'NN'), ('analytics', 'NNS'), ('is', 'VBZ'), ('the', 'D
T'), ('process', 'NN'), ('of', 'IN'), ('extracting', 'VBG'), ('meaningful',
'JJ'), ('information', 'NN'), ('from', 'IN'), ('unstructured', 'JJ'), ('tex
t', 'NN'), ('data', 'NNS'), ('using', 'VBG'), ('natural', 'JJ'), ('language',
'NN'), ('processing', 'NN'), ('techniques', 'NNS'), ('.', '.')]
```

In [9]:
```
# Stopwords Removal
stop_words = set(stopwords.words('english'))
```

```
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]
print("\n3. After Stopword Removal:\n", filtered_tokens)
```

```
3. After Stopword Removal:
 ['Text', 'analytics', 'process', 'extracting', 'meaningful', 'information',
'unstructured', 'text', 'data', 'using', 'natural', 'language', 'processing',
'techniques', '.']
```

In [10]:
```
# Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_tokens]
print("\n4. Stemmed Words:\n", stemmed_words)
```

```
4. Stemmed Words:
 ['text', 'analyt', 'process', 'extract', 'meaning', 'inform', 'unstructur',
'text', 'data', 'use', 'natur', 'languag', 'process', 'techniqu', '.']
```

In [11]:
```
# Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_tokens]
print("\n5. Lemmatized Words:\n", lemmatized_words)
```

```
5. Lemmatized Words:
 ['Text', 'analytics', 'process', 'extracting', 'meaningful', 'information',
'unstructured', 'text', 'data', 'using', 'natural', 'language', 'processing',
'technique', '.']
```

In [12]:
```
# -------------------------------
# TF-IDF REPRESENTATION
# -------------------------------
```

In [13]:
```
from sklearn.feature_extraction.text import TfidfVectorizer

# Sample documents
documents = [
    "Text analytics extracts meaningful information from data",
    "Natural language processing is used in text analytics",
    "Text mining and analytics help understand large text data"
]
```

In [14]:
```
print("\nDocuments:")
for i, doc in enumerate(documents, 1):
    print(f"Doc{i}: {doc}")
```

```
Documents:
Doc1: Text analytics extracts meaningful information from data
Doc2: Natural language processing is used in text analytics
Doc3: Text mining and analytics help understand large text data
```

In [15]:
```
# TF-IDF Vectorization
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)
```

In [16]:
```
# Feature names (words)
features = vectorizer.get_feature_names_out()

print("\nTF-IDF Feature Names:\n", features)
```

```
print("\nTF-IDF Matrix:\n", tfidf_matrix.toarray())
```

```
TF-IDF Feature Names:
 ['analytics' 'and' 'data' 'extracts' 'from' 'help' 'in' 'information' 'is'
 'language' 'large' 'meaningful' 'mining' 'natural' 'processing' 'text'
 'understand' 'used']

TF-IDF Matrix:
 [[0.25712876 0.         0.3311001  0.43535684 0.43535684 0.
  0.         0.43535684 0.         0.         0.         0.43535684
  0.         0.         0.         0.25712876 0.         0.        ]
 [0.22821485 0.         0.         0.         0.         0.
  0.38640134 0.         0.38640134 0.38640134 0.         0.
  0.         0.38640134 0.38640134 0.22821485 0.         0.38640134]
 [0.21826018 0.36954662 0.28104973 0.         0.         0.36954662
  0.         0.         0.         0.         0.36954662 0.
  0.36954662 0.         0.         0.43652037 0.36954662 0.        ]]
```

In [ ]: